

LLD Practice Platform — Research Note

1. Problem Understanding

Low-Level Design (LLD) practice is difficult because solving the problem is only one part of the learning process. A learner must understand requirements, identify important objects and responsibilities, decide where to start, and choose an appropriate way to represent the design.

While practicing an LLD problem, learners commonly ask:

- What are the main requirements?
- Which classes or objects should be created?
- What responsibilities should each class have?
- Should the solution use code, text, or a class/relationship diagram?
- Which abstractions or design patterns are actually useful?
- How can the design be extended when requirements change?
- Is a different solution valid compared with a reference solution?

The biggest challenge is evaluation. Unlike programming problems, LLD problems usually do not have one exact correct solution. Two learners can create different designs and both can be reasonable when their responsibilities, relationships, abstractions, and trade-offs are appropriate.

Existing learning resources can explain LLD concepts and provide examples, but learners can still struggle to understand weaknesses in their own designs. During research, I explored LLD Practice Hub and found its problem-focused practice experience useful as inspiration. This identified an opportunity for a focused platform combining LLD practice with structured, explainable feedback and attempt history.

Main learner problem: Learners can find LLD problems and reference solutions, but need a better way to practice their own designs and understand what they did well, what could be improved, and how to improve in the next attempt.

2. Existing Approaches

Approach 1 — LLD Practice Hub

How it works: Provides LLD problems for practice and lets learners work through requirements and solutions.

Strengths

- Focused specifically on LLD practice.
- Provides problem-based learning.
- Helps learners discover different LLD problem types.
- Useful inspiration for a focused practice experience.

Limitations

- Learners may still have to evaluate their own solutions.
- A reference solution does not always explain why another valid design may work.
- More structured and personalized post-submission feedback is possible.

Approach 2 — Coding Practice Platforms

How it works: Problems have defined inputs, outputs, constraints, and automated evaluation based on predefined test cases.

Strengths

- Clear practice workflow.
- Immediate feedback.
- Repeated attempts.
- Automated evaluation works well when expected output is clearly defined.

Limitations

- LLD usually has no single correct output.
- Test cases cannot sufficiently judge responsibilities, abstractions, coupling, cohesion, or design trade-offs.
- Coding-problem feedback cannot be directly applied to LLD.

Approach 3 — Reference Solutions and Learning Resources

How it works: Learners study articles, tutorials, GitHub repositories, videos, or reference implementations to understand experienced approaches.

Strengths

- Detailed explanations and examples.
- Helps learners understand design patterns and common approaches.
- Useful after attempting a problem.

Limitations

- Learners generally compare their solution manually with the reference.
- A different valid design may appear incorrect.
- Feedback is not personalized to the learner's attempt.
- It does not always support a structured practice → feedback → retry loop.

3. Observed Gaps

Research identified these gaps:

- Learners need guidance from requirements to their own LLD structure.
- Multiple LLD designs can be valid, so evaluation should not depend on one reference solution.
- A single overall score is insufficient; learners need evidence-based feedback.
- Feedback should address responsibilities, abstraction, coupling, cohesion, extensibility, and edge cases.
- Previous attempts should be reviewable so learners can identify improvement over time.
- Future evaluation can support rule-based, AI, or human review.
- The platform should remain focused instead of becoming a complete LMS or large assessment system.

4. Product Direction

The proposed product is a focused LLD practice platform built around:

Choose Problem → Understand → Design → Submit → Get Feedback → Review → Try Again

The platform will provide a small collection of LLD problems through problem cards. Each problem will include its description, requirements, constraints, and relevant context.

A learner starts an attempt and submits a structured text-based design containing:

- Overall approach
- Classes and responsibilities
- Relationships between classes
- Abstractions or interfaces
- Design patterns, if applicable
- Assumptions
- Edge cases

After submission, the platform evaluates the design using a fixed rubric. Deterministic checks handle basic submission validation, while an LLM can evaluate judgment-heavy areas such as responsibility assignment, abstraction quality, extensibility, and design trade-offs.

Instead of returning only a score, feedback includes:

- Criterion score
- Evidence from the submission
- Concerns
- Suggestions for improvement
- Confidence

Previous attempts are stored so learners can review progress and retry the same problem. The MVP uses a simple modular architecture rather than unnecessary distributed infrastructure.

5. MVP Scope

Included

- 3-5 LLD problems with difficulty and context.
- Detailed requirements and constraints.
- Ability to start an attempt.
- Structured text-based solution submission.
- Submission status tracking.
- Deterministic validation.
- Rubric-based evaluation.
- LLM-assisted qualitative feedback.
- Structured evaluation results.
- Attempt history and retry.
- Evaluation-failure handling.
- Tests for important behavior and edge cases.
- Architecture and design documentation.

Submission Format

The MVP uses structured text rather than multiple submission formats. This keeps implementation focused while providing enough information to evaluate important LLD concepts.

Future versions could support code submissions, class diagrams, UML diagrams, and combined text/diagram submissions.

Evaluation

The MVP combines:

- Deterministic evaluation: required fields, submission structure, state, and basic validation.
- LLM evaluation: requirement understanding, responsibility assignment, abstraction and encapsulation, coupling and cohesion, extensibility, edge cases/testability, and improvement suggestions.

The evaluation avoids treating a single reference solution as the only correct answer.

Not Included

The MVP will not attempt to build:

- Complete LMS or complex authentication
- Leaderboards or social features
- Real-time collaboration
- Full UML/diagram editor
- Code execution environment
- Microservices or Kubernetes infrastructure
- Multi-region architecture
- Kafka-based distributed processing
- Complex Redis infrastructure
- Large-scale analytics

These can be considered in future versions if the core practice loop proves useful.